

Connect the context.

Understand the transformer.

Follow the attention weights between words,
then see how transformer blocks build representations.

🕒 20-25 min core reading · 20-30 min practice · 12 short sections

“The movie looked cheerful, but its ending was sad.” To interpret “its”, a model needs information from elsewhere in the sentence. Attention provides a way to combine information across positions. A transformer builds on that operation with multiple heads, feature transformations, and repeated layers.

WHAT YOU WILL UNDERSTAND

The main question: how does one token’s representation gather information from the permitted context?

The practical path: calculate attention weights, inspect a causal mask, combine two heads, and locate this operation inside a transformer.

Reading path: twelve short sections, three diagrams, an interactive attention map, and a Python workbook. Expand the optional notes when you want the mathematical details.

1. Give each position access to useful context

Chapter 23's bigram treated every prefix ending in `is` alike. A recurrent model carries earlier information through a sequence of hidden states. **Self-attention** lets a position combine information directly from other permitted positions in the same sequence.

Consider "I prefer cheerful movies, but tonight I want something sad." The final request depends on more than the nearest word. A learned attention mechanism can help combine such context, but connecting positions is only an operation. Useful behavior still depends on training, the rest of the network, and evaluation.

Attention is not human concentration or a factuality check. It computes input-dependent numerical weights. A large weight says how strongly a particular value vector enters one head's mixture; it is not a complete explanation of the network's reasoning.

2. Understand queries, keys, and values

Imagine comparing the request "a cheerful movie" with descriptions of several films. There are two separate jobs: determine which descriptions match, then gather information from them. The analogy explains the roles, although transformer attention normally operates on vectors at token positions rather than catalog rows.

Vector	Job	Useful question
Query (Q)	Describe the match sought by a position	What information should this position collect?
Key (K)	Provide a representation used for matching	How compatible is this source with that query?
Value (V)	Provide information to combine	What vector should this source contribute?

In self-attention, all three come from the same input representations X , using separate projection matrices: $Q = XW_Q$, $K = XW_K$, and $V = XW_V$. These matrices are normally learned. The same projection is applied at every position within a head; we do not create a new set of parameters for each word. Q, K, and V describe roles, not three separate texts.

➤ Match with keys. Combine the values.

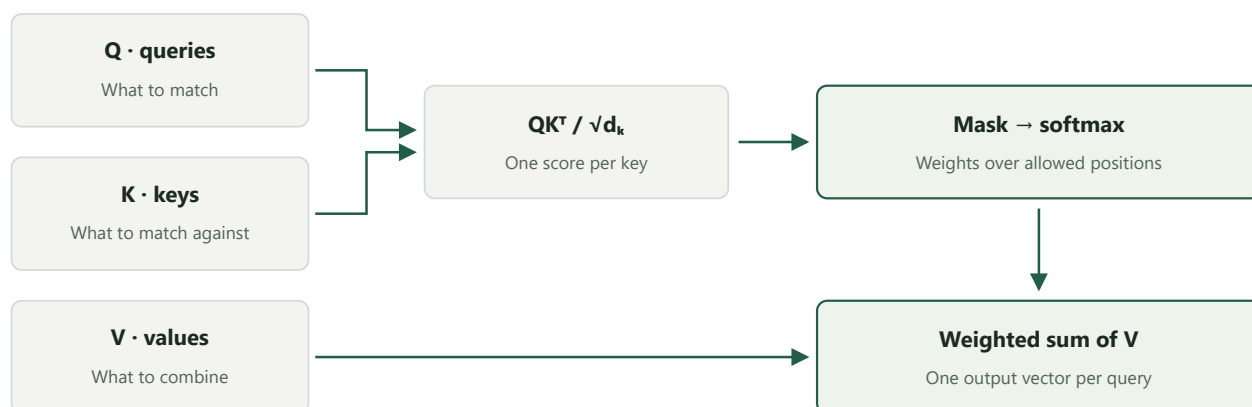


Figure 1. Queries and keys produce compatibility scores. Scaling, masking, and row-wise softmax turn scores into mixing weights. The weighted sum uses values. Its output is a representation, not a distribution over the vocabulary.

3. Calculate one attention head

Scaled dot-product attention follows four steps: compare a query with keys, divide scores by $\sqrt{d_k}$, mask disallowed keys, and apply softmax across the remaining keys. Multiply those weights by the value vectors and sum:

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right), \quad O = AV.$$

d_k is the query/key width. M is zero at allowed positions and negative infinity at blocked positions. Each row of A sums to one before any attention dropout. This is the core operation in *Attention Is All You Need*.

Our example uses `this movie is fun`, with hand-set vectors. In Head 1, the query at `is` is $[1, 1]$. Its dot products with the four keys are $[1, 1, 2, 1]$. Dividing by $\sqrt{2}$ gives approximately $[0.7071, 0.7071, 1.4142, 0.7071]$. Block the future word `fun` before softmax; the weights become **$[0.2483, 0.2483, 0.5035, 0]$** .

The permitted values are $[1, 0]$, $[0, 1]$, and $[0, 0]$. Their weighted sum is **$[0.2483, 0.2483]$** . The largest weight belongs to `is`, yet its zero value contributes nothing in this hand-set example. This shows why a weight alone does not describe the whole computation.

▼ Go deeper: matrix shapes and score scaling

For n tokens of width d_{model} , X has shape $n \times d_{\text{model}}$. A head uses W_Q, W_K of shape $d_{\text{model}} \times d_k$ and W_V of shape $d_{\text{model}} \times d_v$. Consequently, QK^T is $n \times n$, while AV is $n \times d_v$. Softmax runs across **keys within each query row**, not down a column or across the whole matrix.

Why divide by $\sqrt{d_k}$? Under a simplifying assumption of independent, zero-mean, unit-variance query/key coordinates, the dot product has variance d_k . Scaling keeps its variance at one under that assumption, reducing the tendency for large scores to make softmax excessively sharp. It does not force every real score distribution to have unit variance. The workbook uses $d_k = d_v = 2$ and no extra temperature parameter.

4. Control which positions may communicate

A **causal mask** prevents a position from using later inputs. In shifted next-token training, the representation at input `is` predicts `fun`. It may read `this`, `movie`, and `is`, but reading the fourth input `fun` would reveal its target.

The diagonal stays allowed: the current **input** is already known. It is the **next-token target** that must stay hidden. Apply this restriction in every causal attention layer; one unmasked layer could already mix future information into an earlier representation.

➤ **Visibility is part of the computation.**

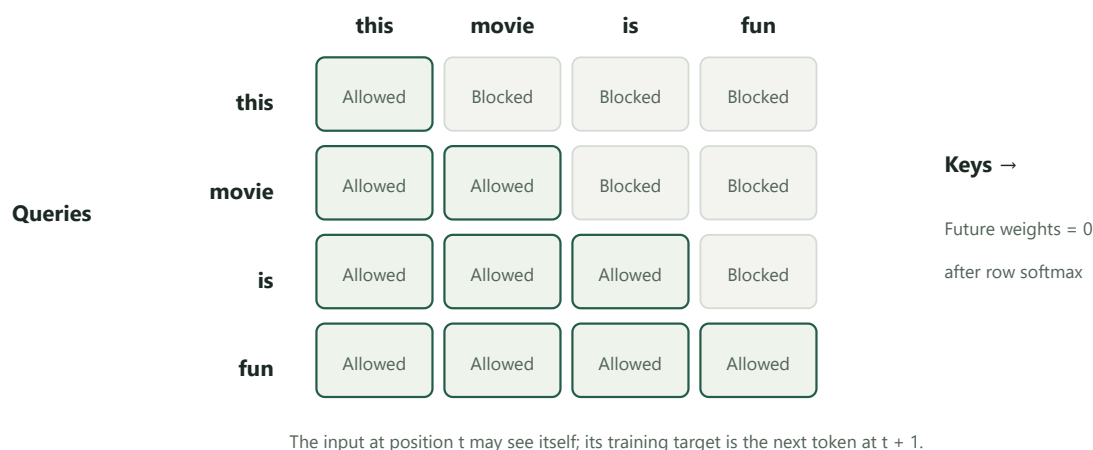


Figure 2. A causal self-attention mask allows the diagonal and earlier keys. The third input, "is", can use "this movie is" to predict the next token, "fun". Reading the fourth input would expose that target.

A **padding mask** has a different purpose: it excludes placeholder positions used to batch different sequence lengths. An encoder may use bidirectional attention among real tokens while still masking padding. Our four-token experiment has no padding.

▼ **Go deeper: mask before softmax, and check the allowed set**

Replacing a blocked score with **zero** is wrong: $\exp(0) = 1$, so the blocked key would still receive weight. A negative-infinity score contributes zero after exponentiation. Our JSON uses `null` to represent that blocked score; the displayed weight is exactly zero.

Setting weights to zero after softmax without renormalizing also changes the result: the surviving weights would no longer sum to one. Every scored row needs a valid policy for its allowed keys. An all-blocked row cannot form an ordinary normalized distribution; this workbook raises an error. Real framework mask conventions differ, so check whether a Boolean value means “allowed” or “blocked” in the API being used.

5. Use multiple heads, then combine their outputs

One set of projections offers one way to compare and mix the inputs. **Multi-head attention** uses several sets in parallel, allowing different learned mixtures. A head is not assigned a guaranteed job such as “genre detector” or “grammar detector”; such labels require evidence from an actual trained model.

Concatenate the head outputs, then apply an output projection:

$$\text{MHA}(X) = \text{Concat}(O_1, \dots, O_h)W_O.$$

With two heads of width 2, concatenation produces width 4. Our 4×4 output projection mixes those features back into a four-feature representation, ready for a residual connection. It is not an average or a vote between heads. D2L’s multi-head explanation develops this construction.

For `is` without positions, our Head 2 has query $[0, 0]$. All allowed dot products are zero, so the three permitted keys receive equal weights. Its output is $[0.6667, 0.6667]$, different from Head 1’s $[0.2483, 0.2483]$. These outcomes follow from the supplied matrices; no head specialization has been learned here.

6. Supply information about position

“The hero follows the villain” and “The villain follows the hero” contain the same words but describe different events. Representations need information about order. Common approaches include learned absolute position embeddings, fixed sinusoidal vectors added to token embeddings, and relative-position mechanisms.

The explorer can add **sinusoidal positional encoding**. With width 4, position zero is $[0, 1, 0, 1]$. Position one is approximately $[0.8415, 0.5403, 0.0100, 1.0000]$. Adding these vectors changes Q, K, and V, even though the words stay the same. The coordinate values are inputs to computation, not human-readable location labels. See D2L on positions.

▼ Go deeper: permutation equivariance and rotary positions

Unmasked self-attention with shared projections and no positional information is **permutation equivariant**: reordering input rows reorders the output rows. It cannot recover an original order that was never represented. A causal visibility pattern already introduces an ordering constraint, so that claim does not apply unchanged when a fixed triangular mask is added.

The workbook uses zero-based position p and feature-pair index i : $PE_{p,2i} = \sin(p/10000^{2i/d})$ and $PE_{p,2i+1} = \cos(p/10000^{2i/d})$, with $d = 4$. We add these to the toy embeddings without scaling the embeddings.

Rotary position embeddings (RoPE) instead rotate query/key coordinate pairs according to position, making their dot products sensitive to relative position. RoPE is not the same as adding our sinusoidal vectors to X . The RoFormer paper describes the method. Positional encoding by itself does not guarantee reliable use of arbitrarily long contexts.

7. Place attention inside a transformer block

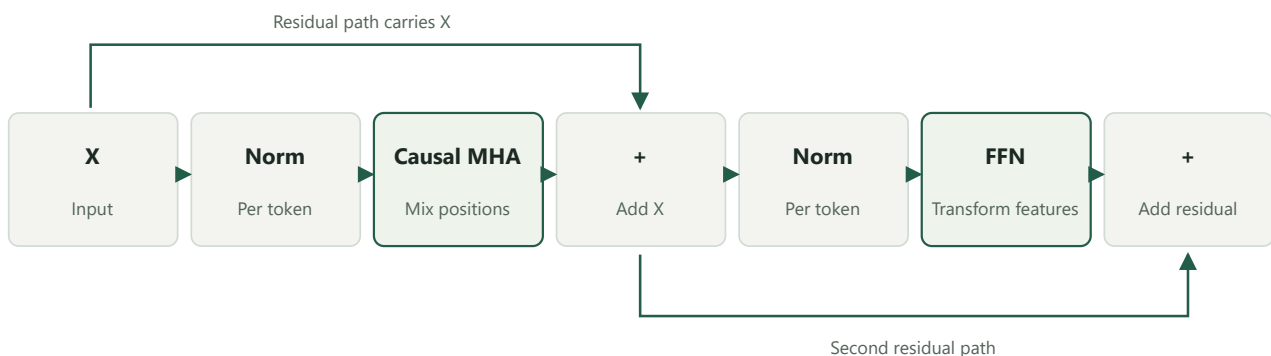
Attention mixes information **between positions**. A **feed-forward network (FFN)** transforms features **at each position**, with weights shared across positions. A **residual connection** adds a sublayer's update to its input, preserving a direct path for information and gradients. **Normalization** controls feature scale within the chosen architecture.

One common pre-normalization layout is:

$$Y = X + \text{MHA}(\text{Norm}(X)), Z = Y + \text{FFN}(\text{Norm}(Y)).$$

Several blocks transform representations repeatedly. A causal language model then uses its final representations to produce vocabulary logits. Chapter 23's softmax and decoding policies act on those logits. The attention softmax mixes **input positions**; the vocabulary softmax predicts **possible output tokens**.

➤ An attention layer is one part of a transformer block.



Stack blocks → final norm → vocabulary logits → decoding

Illustrative pre-norm layout. The original 2017 transformer uses a different normalization order.

Figure 3. A pre-normalization decoder block. Normalize, apply causal multi-head attention, and add a residual; then normalize, apply a position-wise feed-forward network, and add another residual. Stack blocks, then use a final normalization and vocabulary head for next-token scores.

▼ Go deeper: normalization order and the feed-forward network

Layer normalization computes statistics over a token's feature coordinates, rather than combining examples in the batch. It typically includes a learned scale and offset; a small epsilon prevents division by zero. See the [Layer Normalization paper](#). **RMSNorm** is a related alternative based on root-mean-square scaling without mean subtraction.

A simple FFN is $\phi(xW_1 + b_1)W_2 + b_2$, often widening features before projecting back to the residual width. Its nonlinearity makes it more than another linear projection. Some architectures use gated variants. Attention and FFN parameters are normally learned together through the training loss.

The original 2017 transformer used **post-normalization**, normalizing after residual addition; the diagram here shows a pre-normalization variant. The precise norm, ordering, activation, and biases depend on the architecture. Blocks share their operations across positions, but different layers normally have different learned parameter sets. [D2L's transformer chapter](#) walks through the components.

8. Distinguish the main transformer families

Family	Main visibility pattern	Movie-related use
Encoder	Bidirectional self-attention over permitted input positions	Represent a complete review for classification or retrieval
Decoder-only	Causal self-attention over the current sequence	Continue a recommendation explanation token by token
Encoder-decoder	Input encoder; output decoder with causal self-attention and cross-attention	Generate a summary conditioned on a collection of reviews

In **cross-attention**, queries come from one representation sequence, while keys and values come from another. The output still has one row per query. For three output positions attending to five input positions, one head has a 3×5 attention map, not a square one.

Architecture and training objective are related but distinct choices. These families are useful starting points, not an exhaustive list of every visibility pattern or task. Also, **RAG is not synonymous with cross-attention**: a decoder-only RAG application can put retrieved text into its prompt and use ordinary causal self-attention.

9. Separate parallel training from sequential generation

During teacher-forced training, the observed token sequence is available. A causal transformer can calculate representations for many positions together, while its masks prevent access to future targets. Parallel computation does not remove the causal rule.

During ordinary autoregressive generation, the next sampled token is not known in advance. The model processes the prompt, selects a token, and uses it to select the next one. It repeats this process even though each forward pass uses parallel numerical operations. A transformer architecture does not make the entire future answer appear in one independent step.

▼ Go deeper: KV caching and the cost of long context

A **KV cache** stores earlier keys and values at each attention layer during causal inference. A new query can reuse them instead of recalculating the whole prefix. It is runtime state for that sequence, not updated model weights, a vector database, or permanent user memory. Correct caching preserves the intended computation while trading additional memory for less repeated work. See the [Hugging Face cache explanation](#).

Dense self-attention has $n \times n$ query-key pairs per head. At 4 tokens that is 16 score slots; at 8 it is 64. With fixed width, the dense pairwise attention work grows quadratically with sequence length. A causal mask restricts usable pairs to $n(n + 1)/2$, which is still quadratic; implementations differ in whether they calculate or store blocked entries.

Efficient implementations such as [FlashAttention](#) avoid materializing the full attention matrix in high-bandwidth memory and improve data movement. That does not turn dense pairwise attention into a linear-time operation. Chapter 26 will examine efficiency techniques; the current workbook deliberately computes the full tiny matrices for inspection.

10. Explore the attention calculations

Start with Head 1, no positional vectors, and the causal mask enabled. Inspect the `is` row. Change `fun` to `slow`, then repeat with the mask disabled. Switch heads, add positions, and follow the weighted values through the output projection and residual addition.

Same words. Different connections.

Inspect one attention sublayer with two heads. All embeddings and projection matrices are hand-set for teaching; no model is trained.

Default: **this movie is fun**, no positional vectors, causal masking, Head 1. At the third input “is”, $q = [1, 1]$. The four raw dot products are $[1, 1, 2, 1]$; divide by $\sqrt{2}$ and block the fourth key before softmax.

Source	Weight	Value	Weighted value
this	0.2483	[1.0000, 0.0000]	[0.2483, 0.0000]
movie	0.2483	[0.0000, 1.0000]	[0.0000, 0.2483]
is	0.5035	[0.0000, 0.0000]	[0.0000, 0.0000]
fun	0.0000	[1.0000, 0.0000]	[0.0000, 0.0000]

Head 1 output = **[0.2483, 0.2483]**. Head 2 gives equal weight to the three visible keys and outputs $[0.6667, 0.6667]$. Concatenate both heads, multiply by the supplied W_O , and add X to obtain **[1.5816, 1.5816, 0.7908, 0.7908]**.

Replacing “fun” with “slow” cannot change these earlier outputs when causal masking is enabled. The workbook reproduces both heads and all positions. This is attention arithmetic, not a trained transformer or a vocabulary prediction.

The explorer computes attention from hand-set vectors. It does not train embeddings, infer word meanings, or generate a recommendation. Its residual example omits normalization and the FFN to keep this calculation inspectable; use the block diagram for the complete architectural context.

The Python workbook contains four files: `data.json`, `attention.py`, `test_attention.py`, and `README.md`. It uses Python 3.10+ and the standard library, with no model downloads or network calls.

```
py attention.py
py attention.py --ending slow --mask causal
py attention.py --positions sinusoidal --mask bidirectional
py -m unittest -v test_attention.py
```

Use `python` or `python3` if needed. Each calculation command prints a JSON report containing both heads, all query rows, and the combined outputs. Invalid CLI choices exit with code 2. Full-precision reports may differ from rounded screen values.

Exercise 1: reconstruct a weighted sum

Two allowed values are $[2, 0]$ and $[0, 4]$, with attention weights 0.25 and 0.75. What is the output vector? Does 0.75 mean the second word is 75% likely to be generated?

▼ Reveal the reasoning

The output is $0.25[2, 0] + 0.75[0, 4] = [0.5, 3]$. The coefficient describes mixing a source value into a representation. It is not a probability over the output vocabulary.

Exercise 2: find a masking error

You want the input `is` to predict `fun`. A developer blocks later positions by replacing their scores with zero before softmax. Why can the target still leak into the representation? Should the `is` diagonal also be blocked?

▼ Reveal the reasoning

A zero score still has exponential value 1, so future values receive nonzero weight. Exclude those keys before normalization, equivalently assigning a negative-infinity score. Keep the diagonal: "is" is the known input; "fun" is the next-token target.

Exercise 3: trace shapes and cost

There are 8 input tokens of width 12 and 3 heads, each with query, key, and value width 4. What are the shapes of one attention map, one head output, the concatenated output, and W_O ? How many dense score slots are needed across all heads, and what happens at 16 tokens?

▼ Reveal the reasoning

The shapes are 8×8 , 8×4 , 8×12 , and 12×12 . There are $3 \times 8 \times 8 = 192$ dense score slots. At 16 tokens there are $3 \times 16 \times 16 = 768$, four times as many. A causal mask changes which pairs are allowed, not the square shape used in this calculation.

11. Check your understanding

01 Which vectors supply the information combined into one attention output?

- A. The values, weighted using query-key compatibility.
- B. Only the queries; the values are labels.
- C. The next-token vocabulary probabilities.

Answer: A. Q and K determine mixing weights; those weights multiply V. The attention output is a feature vector, before the vocabulary prediction head.

02 At the input "is" in "this movie is fun", which positions may causal self-attention use?

- A. Only "this" and "movie"; the diagonal must be blocked.
- B. "this", "movie", and "is"; the target "fun" is still hidden.
- C. All four words because training already knows the answer.

Answer: B. The target is shifted by one position. An input may see itself and the earlier prefix, but cannot use its next-token target or anything later.

03 Two attention heads each output 2 features per token. What happens before adding a 4D residual?

- A. Keep only the larger head.
- B. Average the heads into 2 features and add them directly.
- C. Concatenate to 4 features, then apply the output projection.

Answer: C. Concatenation preserves both outputs. The output projection mixes them and must return the residual stream's width. The teaching model uses a 4×4 output projection.

04 Why can causal transformer training process several positions together?

- A. It is allowed to reveal future target tokens.
- B. The observed input sequence is available, while a mask limits each position's dependencies.
- C. Autoregressive generation also knows all future sampled tokens.

Answer: B. Teacher forcing supplies the observed sequence. Causal masking prevents future access within the parallel computation. Ordinary generation still needs the previously chosen tokens before choosing the next one.

05 A cell in the attention map has weight 0.8. What can you conclude?

- A. The source word is 80% likely to be the next generated word.
- B. The model's answer is 80% factually correct.
- C. That head assigns the source value a mixing coefficient of 0.8 for this query.

Answer: C. The map is over input positions, not vocabulary outcomes or factual truth. Other heads, projections, residual paths, and later layers also affect the result.

12. Connect the architecture to LLM training

You can now trace the mechanism: **represent tokens** → **form Q, K, V** → **score and mask** → **mix values** → **combine heads** → **update features through transformer blocks**. An attention map is one intermediate calculation in that chain.

Chapter 25: How LLMs are trained and generate text will connect this architecture to pretraining data, training objectives, adaptation, and the complete generation process. A network's architecture supplies a computation; its training and application determine how useful that computation becomes.

▼ Quick-reference glossary

Term	Meaning
Self-attention / cross-attention	Q, K, V from one sequence / queries and key-values from different sequences
Query / key / value	Matching request / matching representation / information to combine
Head / output projection	One set of attention projections / mixing concatenated head features
Causal mask / padding mask	Prevent future access / exclude placeholder content
Positional encoding / RoPE	Supply position information / rotate queries and keys by position
Residual connection	Add a sublayer update to an existing representation
FFN / normalization	Per-position nonlinear feature transform / control feature scale
Pre-norm / post-norm	Normalize before a sublayer / after residual addition
KV cache	Reusable per-layer keys and values during sequence inference
Vocabulary head	Map final hidden features to candidate-token scores

Sources and further reading

References are linked beside the concepts: Attention Is All You Need; D2L's attention, position, and transformer chapters; Layer Normalization; RoFormer; the Hugging Face cache guide; and FlashAttention. The movie examples, fixed matrices, diagrams, workbook, and worked calculations are original teaching material.